FleetSieve: Decision-Critical Profiling for SLO-Aware LLM Fleet Configuration

Huang Cheng Scott Zhang Aubert Li
Meta, Menlo Park, California, USA
huangcheng@meta.com

Abstract

Choosing tensor-parallel (TP) degrees and replica counts for an LLM serving fleet is difficult because performance is not monotonic in TP and the feasible choice can change with load. Exhaustive profiling resolves this uncertainty, but measures many configurations that do not affect the final resource allocation.

We present FleetSieve, which selects measurements according to their expected effect on a resource-coupled, SLO-aware fleet decision. FleetSieve models capacity and tail latency jointly, compares conservative and optimistic allocations, and stops when their remaining decision gap is below a specified tolerance. On a fixed H100 measurement grid for a 31B-parameter open-weight model, FleetSieve reaches the oracle aggregate decision using 22,200 GPU-seconds, 6.9% less than uniform random profiling in the fixed comparison. Across 200 random reveal orders, its mean saving over random profiling is 5.4% (95% bootstrap CI: 3.5–7.2%). The fixed-comparison saving is 21.5% for Chat, while FleetSieve does not use the fewest GPU-seconds for Code. Joint capacity and tail modeling also avoids selecting a configuration whose 46.4-second completion p99 violates a 30-second SLO. In a 16-GPU allocation, an incorrect sparse-profile decision loses up to 1.93 requests/s and 12.4 percentage points of max-min fulfillment. Boundary repeats and BurstGPT measurements support the observed load-dependent tail-latency mechanism.

1 Introduction

An LLM serving system rarely serves one homogeneous request stream. Interactive chat, code assistance, ranking, and background generation differ in arrival process, token lengths, and latency objectives. Under a finite GPU resource budget, an operator must choose a parallel configuration and a replica count for each class. These choices interact: assigning more GPUs to one replica may reduce its latency and increase its memory headroom, but it also leaves fewer GPUs for replicas serving other workloads.

Tensor parallelism illustrates this tension. Increasing TP shards model state over more GPUs and can improve per-request latency and KV-cache headroom, but it increases collective communication and reduces the number of replicas that fit in the same fleet. Consequently, neither *always use the smallest TP* nor *always use the largest TP* is generally correct. In our measurements, the SLO-feasible throughput per GPU peaks at the interior choice TP4 for both workloads. Under heavier chat load, however, TP4 violates the completion-time objective while TP8 remains feasible. Thus the relevant target is not a context-free TP ranking; it is the final resource-constrained fleet decision.

The conventional solution is to profile every candidate. This is reliable but scales with the Cartesian product of models, TP degrees, request shapes, loads, and serving policies. Generic active-learning and Bayesian-optimization methods reduce measurements, but they commonly target predictive uncertainty or the best isolated configuration. A fleet planner instead needs to know whether another measurement can change a coupled allocation.

FleetSieve treats profiling as downstream decision identification. It maintains uncertainty over capacity and tail latency, solves the fleet allocation under conservative and optimistic realizations, and measures configurations that are expected to reduce disagreement between those decisions. It stops only when critical feasibility is resolved and the remaining allocation gap is below a tolerance.

This paper makes three contributions:

1. **Decision-critical acquisition.** Profiling is driven by a measurement's predicted effect on the downstream resource-coupled allocation rather than by grid coverage or marginal uncertainty alone.
2. **Joint capacity and tail modeling.** Capacity and the binding tail metric share the decision path, preventing a high-throughput but tail-infeasible configuration from being certified.
3. **A conditional decision certificate.** The profiler returns a three-state result—certified feasible, undecided, or certified infeasible—under an empirical uncertainty model. A replay that constructs priors from revealed measurements only produces the same stopping point and allocation.

Our evaluation covers one 31B model and one H100 platform. FleetSieve reduces aggregate profiling GPU-seconds modestly, provides a larger reduction for Chat, and provides no reduction for Code. We therefore interpret the method as selectively useful when unresolved measurements can still change the allocation, rather than

as a universal improvement.

2 Background and Related Work

LLM serving. PagedAttention in vLLM [1] and continuous batching in Orca [2] improve memory utilization and scheduling. Sarathi-Serve [3], DistServe [4], Splitwise [5], and AlpaServe [6] optimize batching, phase separation, or placement. SLOs-Serve [7] allocates serving resources across multiple objectives, while Nitsum [8] adapts TP at runtime for tiered requests. These systems motivate a rich configuration space; FleetSieve addresses the complementary question of which measurements are needed before committing a fleet configuration.

Configuration search and decision-focused measurement. SCOOT [9] applies constrained Bayesian optimization to LLM inference-engine tuning. Active learning more generally chooses informative samples [10]; targeted active learning explicitly optimizes information about a downstream Bayesian decision [11]. Fixed-confidence combinatorial exploration, such as CombGapE [12], identifies an optimal combinatorial action with few samples. FleetSieve borrows this decision-focused perspective but instantiates it for correlated serving measurements, multiple SLO metrics, and an integer, resource-coupled fleet allocation. We do not claim that active learning, Bayesian optimization, elimination, or fixed-confidence stopping is independently new.

Public traces. Our primary replay uses the public Azure LLM inference traces accompanying DynamoLLM [13]; the dataset is distributed under CC BY. BurstGPT [14] supplies an independent conversation trace for a mechanism-level robustness check.

Resource-constrained selection. Resource-constrained selection also appears in other infrastructure domains. Cheng et al. [15], for example, formulate continuous roadside coverage as a knapsack-constrained Steiner-tree problem. FleetSieve addresses a different system and uses a different algorithm, but shares the broader objective of concentrating a finite resource budget on decisions that determine system-level utility.

3 Problem Formulation

Let workload class $i \in \mathcal{W}$ have offered demand λ_i, an SLO L_i, and priority p_i. Priorities define a set of critical classes $\mathcal{C}$ and minimum fulfillment floors δ_i for $i \in \mathcal{C}$. A serving configuration $q \in \mathcal{Q}_i$ specifies a TP degree and an admission/load setting. It consumes g_q GPUs per replica. Its unknown performance vector is

$$\theta_{iq} = (c_{iq}, \ell_{iq}, s_{iq}),$$

where c_{iq} is sustainable throughput, ℓ_{iq} is the relevant tail-latency metric, and s_{iq} is success probability. A configuration is feasible only when its tail and success metrics satisfy the class policy.

The allocator chooses an integer replica count n_{iq} and served load x_i. In our implementation each class uses one homogeneous configuration, expressed with a binary selector y_{iq}:

$$\sum_q y_{iq} = 1, \qquad n_{iq} \le M y_{iq}, \tag{1}$$

$$x_i \le \sum_q n_{iq} c_{iq}, \tag{2}$$

$$\sum_{i,q} n_{iq} g_q \le G, \tag{3}$$

$$0 \le x_i \le \lambda_i, \quad n_{iq} \in \mathbb{Z}_{\ge 0}, \quad y_{iq} \in \{0,1\}, \tag{4}$$

$$y_{iq} = 1 \Rightarrow \ell_{iq} \le L_i,\ s_{iq} \ge s_i^{\min}. \tag{5}$$

The satisfaction ratio is $r_i = x_i/\lambda_i$. The objective is lexicographic: (1) enforce $r_i \ge \delta_i$ for critical classes whenever feasible, (2) maximize $\min_i r_i$, (3) maximize total served goodput, and (4) minimize fragmentation. Profiling uses a separate compute budget measured in GPU-seconds.

If all θ_{iq} were known, exhaustive profiling would produce the oracle allocation. The profiling problem is to approach that decision while revealing only a subset of the table.

4 FleetSieve

4.1 Joint uncertainty state

After round t, FleetSieve maintains capacity intervals $[\underline{c}_{iq,t}, \overline{c}_{iq,t}]$ and tail intervals $[\underline{\ell}_{iq,t}, \overline{\ell}_{iq,t}]$. The intervals combine an interior-knee structural prior with measured residuals. They are empirical rather than distribution-free; Section 6.4 describes the audit that bounds this claim.

The conservative allocation A_t^- uses lower capacity and upper tail bounds. The optimistic allocation A_t^+ uses upper capacity and lower tail bounds. Both use the same GPU constraint, SLO policy, and lexicographic objective.

4.2 Decision certificate

The state is CERTIFIED-INFEASIBLE if even A_t^+ cannot meet the critical floor. It remains UNDECIDED if critical feasibility is unresolved or the optimistic and conservative objectives differ by more than ε. It is CERTIFIED-FEASIBLE when A_t^- satisfies the critical policy, every committed tail upper bound is known and within SLO, and the stage-wise allocation gap is at most ε.

This certificate is conditional on the uncertainty set containing the relevant performance table. We therefore use "empirical" or "conditionally calibrated," not an unconditional correctness guarantee.

Algorithm 1 FleetSieve profiling loop

Initialize a common sparse design for every method.
while unmeasured candidates remain **do**
  Fit joint capacity and tail intervals.
  $A^- \leftarrow$ conservative fleet allocation.
  $A^+ \leftarrow$ optimistic fleet allocation.
  $z \leftarrow \textsc{Certificate}(A^-, A^+, \varepsilon)$.
  if $z = \textsc{Certified-Feasible}$ **then**
    return A^-.
  else if $z = \textsc{Certified-Infeasible}$ **then**
    return Infeasible.
  end if
  Reveal the candidate with maximum decision-gap reduction.
end while
return the best measured feasible allocation.

4.3 Acquisition

For each unmeasured experiment e, FleetSieve estimates how revealing its outcome would shrink capacity/tail uncertainty and change the conservative–optimistic allocation gap:

$$\text{score}_t(e) = \widehat{\mathbb{E}}[\Gamma_t - \Gamma_{t+1} \mid e],$$

where Γ_t is the stage-wise downstream allocation gap. The implementation uses a deterministic approximation to this value-of-information objective. Expected profiling GPU-seconds are a strict tie-breaker, not a divisor; consequently we describe the rule as decision-critical rather than universally compute-resource-optimal.

5 Experimental Methodology

System. We evaluate a 31B-parameter open-weight decoder model in FP8 on a single H100 node, using a vLLM-based serving stack with speculative decoding disabled. Candidate TP degrees are $\{2, 4, 8\}$. Each cell runs for 300 seconds; profiling is measured as duration multiplied by GPUs used, reported in GPU-seconds.

Workloads and SLOs. The primary workload uses the public Azure conversation and code traces [13]. Trace records provide arrival times and input/output token counts, not prompt content. The chat policy requires success $\geq 99\%$, TTFT-p99 ≤ 2 seconds, and completion-p99 ≤ 30 seconds. The code policy requires success $\geq 99\%$ and TTFT-p99 ≤ 5 seconds. Load labels such as C96 are ordinal indices into a documented arrival-scale ladder, not literal in-flight request counts.

The fixed primary grid contains 21 standardized cells plus four earlier code-TP2 calibration cells. The four cells use a different warm-up split and remain separately labeled; excluding them does not change the code oracle because TP2 is dominated. Boundary experiments repeat eight TP4/TP8 cells over five seeds. BurstGPT adds 18 measured cells: three load scales, two TP degrees, and three seeds.

Table 1: Peak SLO-feasible throughput (requests/s/GPU).

Workload	TP2	TP4	TP8
Chat	0.798	**1.649**	1.408
Code	0.560	**2.779**	1.392

Baselines and metrics. All methods start from the same observations, use the same candidate table, SLOs, allocator, and empirical stop. We compare uniform random, fixed grid order, joint maximum uncertainty, a shared-feature uncertainty surrogate, targeted active learning, value of information, constrained Bayesian optimization, and FleetSieve. Primary metrics are GPU-seconds to an ε-correct certified decision ($\varepsilon = 0.05$), decision regret versus the fully measured oracle, and regret AUC. We also report the resulting 16-GPU allocation, boundary stability, and tail-certificate controls.

6 Results

6.1 The TP optimum is interior and load dependent

Table 1 and Figure 1 report peak SLO-feasible throughput per GPU. Both workload classes peak at TP4; neither smaller nor larger TP is uniformly best.

Across five repeats at the heavier Chat C128 point, TP4 and TP8 both serve 11.27 requests/s. Their mean completion-p99 values are 46.4 and 25.2 seconds, respectively, so only TP8 passes the 30-second objective. Thus the SLO-feasible TP can change even when throughput does not.

6.2 Decision-critical profiling is modestly better in aggregate

Table 2 and Figure 2 report GPU-seconds to the common empirical certificate. On the fixed aggregate comparison, FleetSieve uses 22,200 GPU-seconds, 6.9% below random and 9.8–21.3% below the other listed baselines. Every deterministic run and every random trial reaches zero regret; the oracle TP is 4 for both classes.

The class split is important. Against random, the fixed Chat comparison saves 21.5%; against the remaining baselines it saves 30.4–38.5%. Its Chat regret AUC is 0.1256, about half the fixed-grid value of 0.2459. For Code, random, shared-feature uncertainty, and joint uncertainty use fewer GPU-seconds than FleetSieve because TP4 becomes evident early.

The table value and the randomized estimate answer different questions. Across 200 random reveal orders, the ratio-of-means advantage over random is 5.4%, with a

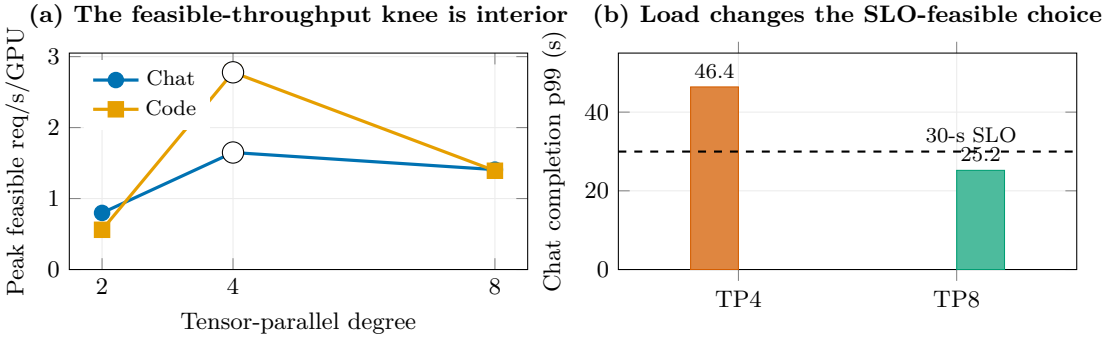

Figure 1: Why profiling is necessary. (a) Peak SLO-feasible throughput per GPU is non-monotone and peaks at TP4 for both Azure workload classes. (b) At the heavier Chat C128 point, TP4 and TP8 serve the same raw request rate, but only TP8 satisfies the completion-time objective; bars show the five-seed mean.

Table 2: GPU-seconds to the same zero-regret certified decision. Lower is better. One row groups three methods that are equivalent on this fixed grid: targeted active learning, value of information, and constrained Bayesian optimization.

Method	Aggregate	Chat	Code
FleetSieve	**22,200**	**9,600**	12,600
Uniform random	23,844	12,228	11,616
Shared-feature uncertainty	24,600	14,400	**10,200**
Joint maximum uncertainty	25,800	14,400	11,400
Fixed grid order	26,400	13,800	12,600
Targeted AL / value-VOI / constrained BO	28,200	15,600	12,600
Full grid	29,400	–	–

95% bootstrap CI of 3.5–7.2%. For Chat it is 19.4% (16.2–22.3%). On a single random draw, FleetSieve uses fewer GPU-seconds only 58% of the time in aggregate and 65% for Chat. The gain is reliable in expectation, not for every ordering. A 500-replicate parametric bootstrap over the repeated boundary cells never changes the oracle decision; this noise analysis covers the repeated cells and holds the remaining grid cells fixed.

A heuristic early stop based on a fixed 1.5× ceiling is unsafe: fixed-grid and joint-uncertainty profiling stop with TP8 on Chat and regret 0.2407. Under the common empirical certificate, all methods continue to the correct TP4 frontier. This failure motivates evaluating the stopping rule rather than hindsight "first correct" GPU-seconds.

6.3 Profiling changes a constrained 16-GPU allocation

We allocate 16 GPUs between Chat and Code at three demand scales (Table 3 and Figure 3). The fully measured oracle and FleetSieve choose two TP4 replicas per class. A sparse independent rule chooses one TP8 Chat replica and two TP4 Code replicas. Both consume eight GPUs per class, but the Chat capacity differs.

At 0.7× demand, spare capacity masks the decision and all three serve the full load. At 1.0×, the sparse decision loses 0.73 requests/s and 6.1 percentage points of max-min fulfillment. At 1.3×, the losses grow to 1.93 requests/s and 12.4 points. This is the resource-coupled consequence that an isolated TP-accuracy metric misses. Because every row uses the same 16 GPUs, we do not report a GPU-count saving.

6.4 Tail latency is load-bearing in the certificate

A capacity-only rule selects Chat TP4 at C128 because it serves 11.27 requests/s, but its 46.4-second completion-p99 violates the 30-second objective. The joint rule selects TP4 at C96 for the frontier decision, where completion-p99 is 15.98 seconds, and uses TP8 when C128 must be served. The certificate refuses to fire until the committed configuration's tail upper bound is known.

A separate fixed-order audit exercises the state machine. A non-critical trajectory certifies at step 20 (27,000 GPU-seconds); a trajectory with critical floors of 6 requests/s for Chat and 10 requests/s for Code certifies at step 17 (22,800 GPU-seconds); an intentionally infeasible 1,000-requests/s demand returns CERTIFIED-INFEASIBLE. These GPU-seconds audit the certificate and are not the acquisition-comparison GPU-seconds in Table 2.

The initial offline audit used an optimistic prior derived from unobserved peak cells early in the trajectory. To test whether it affected the result, we replaced that branch with a revealed-only prior and replayed the audit. The branch is inactive after the initial six probes, and both versions

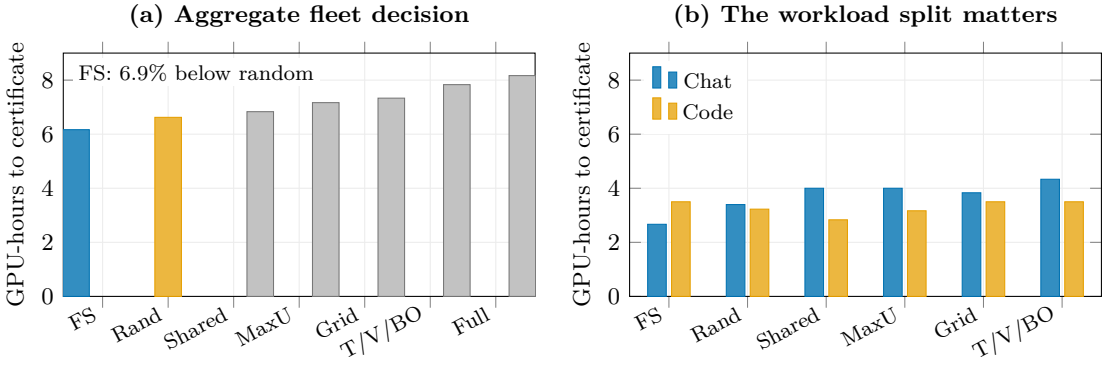

Figure 2: Profiling GPU-seconds under the common empirical certificate. FS denotes FleetSieve; Rand is uniform random; Shared is the shared-feature uncertainty surrogate; MaxU is joint maximum uncertainty; T/V/BO groups three methods that are equivalent on this grid. FleetSieve is modestly better in aggregate, substantially better for Chat, and not better for Code.

Table 3: Effect of the profiling decision on the 16-GPU allocation. MM is max-min fulfillment; goodput is served requests/s. The sparse rule chooses Chat TP8 rather than the oracle TP4.

Demand	Method	TP (Chat/Code)	Replicas	MM	Goodput
0.7×	Oracle / FleetSieve	4 / 4	2 / 2	1.000	19.60
	Sparse rule	8 / 4	1 / 2	1.000	19.60
1.0×	Oracle / FleetSieve	4 / 4	2 / 2	1.000	28.00
	Sparse rule	8 / 4	1 / 2	0.939	27.27
	Fixed TP8	8 / 8	1 / 1	0.696	22.40
	Fixed TP2	2 / 2	3 / 5	0.350	10.39
1.3×	Oracle / FleetSieve	4 / 4	2 / 2	0.846	33.99
	Sparse rule	8 / 4	1 / 2	0.722	32.07

certify at the same step, GPU-seconds, and allocation. We label the certificate empirical and conditionally calibrated; an online implementation should use the revealed-only path.

6.5 Repeated boundaries and a second public trace

Across five seeds on eight boundary cells, seven cells retain the same feasibility outcome. Chat TP4 at C96 is the exception: it satisfies the policy in four of five runs because one run falls to 94.3% success. The load-bearing C128 distinction is stable in every run: TP4 fails with a mean completion-p99 of 46.38 seconds, while TP8 passes at 25.15 seconds.

BurstGPT reproduces the mechanism direction: TP8 has lower completion-p99 at all three tested loads, and the gap grows with load (Figure 4). It does *not* reproduce a uniquely feasible TP8 cell: both TPs pass at scale 250 and both fail at 375. The trace's absolute timescale is compressed 250–500×, so this is a load-controlled robustness check, not a timescale-faithful operational replay or a second end-to-end acquisition win.

7 Discussion and Limitations

The evaluation covers one 31B-parameter model, one serving stack, and one H100 platform. Model architecture, quantization, interconnect, and runtime can change the balance between collective-communication time and KV-cache capacity, and may move or remove the observed interior optimum. BurstGPT broadens the token-length and burst distribution, but tests the tail-latency mechanism rather than the complete acquisition algorithm.

The certificate relies on empirically calibrated residual bands and an interior-knee structural assumption. A multimodal or sharply discontinuous capacity curve could violate these assumptions. Measurement noise also matters near an SLO boundary: one of eight repeated cells changes feasibility across seeds, although the C128 cell that changes the TP choice is stable in every repeat.

The primary grid includes four TP2 calibration cells collected with a different warm-up split. They remain labeled separately, and removing them does not change the oracle because TP2 is dominated on the reported metric. Finally, the fleet experiment fixes the GPU resource allocation at 16. It improves served demand and fairness without reducing the steady-state GPU count.

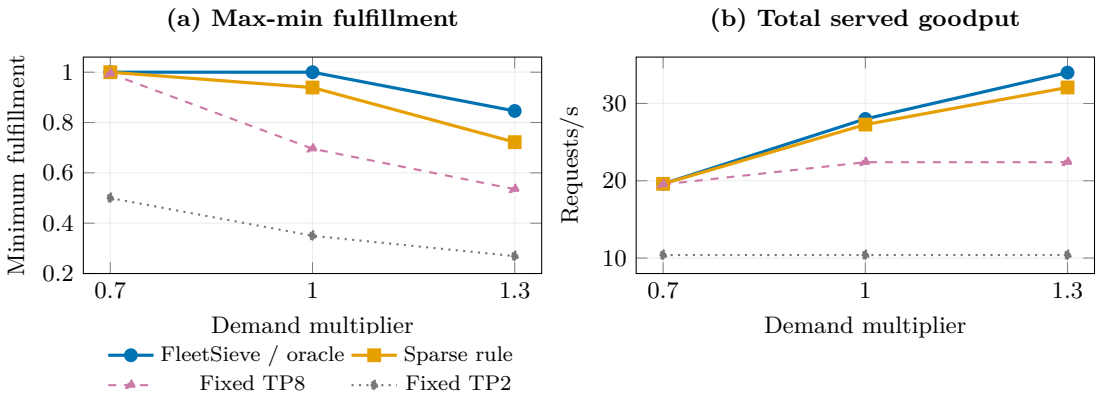

Figure 3: A profiling decision changes the resource-coupled fleet outcome. At low demand, spare capacity hides the difference. As demand rises, the sparse TP8/TP4 choice loses both fairness and served load relative to FleetSieve and the fully measured oracle. Every method uses the same 16-GPU resource allocation.

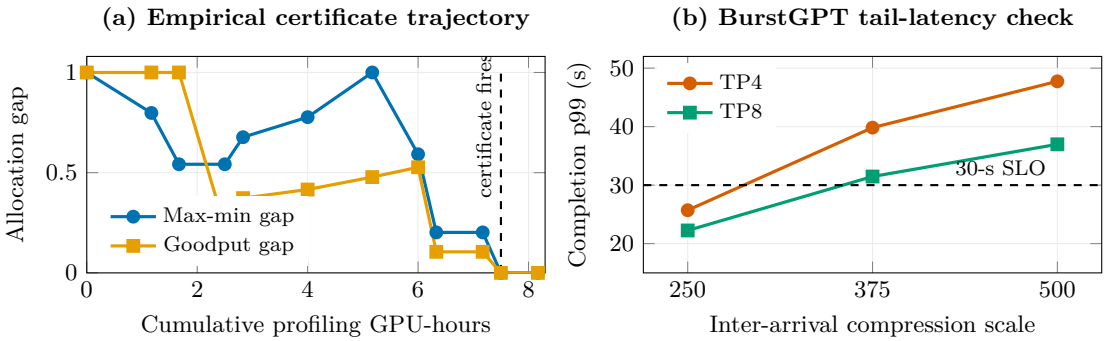

Figure 4: Uncertainty and robustness. (a) A fixed-order audit shows that the conservative and optimistic fleet objectives converge only after the decisive tail measurement; the non-monotonic gaps reflect changes in the optimistic allocation as new cells are revealed. (b) On BurstGPT, TP8 has lower completion p99 at every tested load, but the three-point grid does not contain a TP8-only feasible point.

8 Conclusion

FleetSieve profiles an LLM serving configuration space according to the fleet allocation that the measurements inform. On the evaluated H100 grid, it reaches the oracle aggregate decision using 22,200 GPU-seconds, 6.9% less than uniform random profiling in the fixed comparison. The benefit is larger for Chat and absent for Code. Joint capacity and tail modeling prevents a measured 46.4-second SLO violation, while the resulting TP choice recovers up to 1.93 requests/s and 12.4 percentage points of max-min fulfillment in a 16-GPU fleet. These results show that decision-aware profiling can reduce measurement effort when uncertainty affects the allocation, while its value remains workload- and calibration-dependent.

A Reproducibility Protocol

Each standardized primary cell uses a 30-second warm-up and a 270-second measurement window. The four earlier calibration cells use a 10-second warm-up and 290-second measurement window and are never silently mixed with standardized repeats. Run metadata include model precision, TP, trace window, arrival scale, warm-up, measurement duration, seed, success rate, TTFT, completion time, and failure state. Raw records are immutable; derived feasibility is recomputed from the class policy.

The initial observations, candidate set, per-cell GPU-seconds, allocator, SLO policy, and oracle table are identical across all profiling methods. Random profiling is evaluated over multiple reveal orders. Failed, rejected, and timed-out requests do not count as SLO goodput.

B Baseline Definitions

- **Uniform random** samples remaining cells uniformly.
- **Fixed grid** traverses a predefined TP/load order.
- **Joint maximum uncertainty** selects the largest marginal uncertainty from the shared model without decision impact.

- **Shared-feature uncertainty** uses an RBF cross-TP surrogate and selects by uncertainty.
- **Targeted active learning** prioritizes information about the current decision.
- **Value of information** estimates expected decision improvement with its own compute-resource normalization.
- **Constrained Bayesian optimization** uses independent per-arm models, expected improvement, and feasibility constraints.

References

[1] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of SOSP*, 2023.

[2] G.-I. Yu et al. Orca: A distributed serving system for transformer-based generative models. In *Proceedings of OSDI*, 2022.

[3] A. Agrawal et al. Sarathi-Serve: Efficient LLM serving with chunked prefills. In *Proceedings of OSDI*, 2024.

[4] Y. Zhong et al. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of OSDI*, 2024.

[5] P. Patel et al. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of ISCA*, 2024.

[6] Z. Li et al. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *Proceedings of OSDI*, 2023.

[7] S. Chen, Z. Jia, S. Khan, A. Krishnamurthy, and P. B. Gibbons. SLOs-Serve: Optimized serving of multi-SLO LLMs. arXiv:2504.08784, 2025.

[8] V. Srivatsa, Z. He, P. Guo, D. Li, and Y. Zhang. Nitsum: Serving tiered LLM requests with adaptive tensor parallelism. arXiv:2605.05467, 2026.

[9] K. Cheng, Z. Wang, W. Hu, T. Yang, J. Li, and S. Zhang. SCOOT: SLO-oriented performance tuning for LLM inference engines. In *Proceedings of The Web Conference*, 2025.

[10] B. Settles. *Active Learning*. Morgan & Claypool, 2012.

[11] L. Filstroff, I. Sundin, P. Mikkola, A. Tiulpin, J. Kylmäoja, and S. Kaski. Targeted active learning for Bayesian decision-making. arXiv:2106.04193, 2021.

[12] S. Nakamura and M. Sugiyama. A fast algorithm for the real-valued combinatorial pure exploration of multi-armed bandit. arXiv:2306.09202, 2023.

[13] J. Stojkovic, C. Zhang, I. Goiri, J. Torrellas, and E. Choukse. DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. In *Proceedings of HPCA*, 2025.

[14] Y. Wang et al. BurstGPT: A real-world workload dataset to optimize LLM serving systems. In *Proceedings of KDD*, 2025.

[15] H. Cheng, X. Fei, M. Almulla, and A. Boukerche. A knapsack constrained Steiner tree model for continuous coverage over urban VANETs. In *Proceedings of IEEE ICC*, pages 130–135, 2014.